

# What is CRUD?

CRUD is an acronym for the four basic operations performed on data in a database.

| Letter | Meaning | Implemented using | Category | Notes                                |
|--------|---------|-------------------|----------|--------------------------------------|
| C      | Create  | INSERT            | DML      | Because you're modifying table data. |
| R      | Read    | SELECT            | DQL      | You're only retrieving data.         |
| U      | Update  | UPDATE            | DML      |                                      |
| D      | Delete  | DELETE            | DML      |                                      |

```
CREATE TABLE Student(...)
```

## Is this CRUD?

No.

It creates a table, not a record.

CRUD deals with rows of data, not the database structure.

## Step 1 — Create

Suppose a new student joins.

```
INSERT INTO Students
VALUES (1, 'Aadya', 20);
```

Row created.

## Step 2 — Read

---

Teacher searches for the student.

```
SELECT *  
FROM Students;
```

Row retrieved.

## Step 3 — Update

---

Student changes age.

```
UPDATE Students  
SET Age = 21  
WHERE StudentID = 1;
```

Row modified.

## Step 4 — Delete

---

Student record is removed.

```
DELETE FROM Students  
WHERE StudentID = 1;
```

Row deleted.

## Create the database:

```
CREATE DATABASE CollegeDB;  
USE CollegeDB;
```

## Students Table

```
CREATE TABLE Students(  
  StudentID INT AUTO_INCREMENT PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Age TINYINT UNSIGNED,  
  Gender ENUM('Male', 'Female', 'Other'),  
  Department VARCHAR(50),  
  CGPA DECIMAL(3,2),  
  Email VARCHAR(255) UNIQUE,  
  AdmissionDate DATE  
);
```

## INSERT Basics

```
INSERT INTO Students  
VALUES (1, 'Aadya', 20, 'Female', 'ECE', 7.97, '24uec006@gmail.com', '2025-08-20');
```

One new row is inserted.

## General Syntax

### Method 1 (All Columns)

```
INSERT INTO table_name  
VALUES (...);
```

### Method 2 (Recommended)

```
INSERT INTO table_name(col1,col2,...)  
VALUES (...);
```

## Inserting Specific Columns

This is the most common style in production.  
Example:

```
INSERT INTO Students(Name, Age)
VALUES ( 'Aadya' , 20 );
```

Only Name and Age are provided.

All other columns receive:

- their DEFAULT value,
- NULL,
- or an AUTO\_INCREMENT value (if applicable).

## 2.4 Multiple Row Insert

Instead of:

```
INSERT INTO Students(Name, Age)
VALUES( 'Aadya' , 20 );

INSERT INTO Students(Name, Age)
VALUES( 'Arpit' , 21 );
```

Write:

```
INSERT INTO Students(Name, Age)
VALUES
( 'Aadya' , 20 ),
( 'Arpit' , 21 ),
```

**Advantages:**

- Faster
- Less network overhead
- Fewer transactions

## Timestamp Defaults

```
CREATE TABLE LoginHistory(
    ID INT AUTO_INCREMENT PRIMARY KEY,
    LoginTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Insert:

```
INSERT INTO LoginHistory  
VALUES(NULL,DEFAULT);
```

LoginTime is automatically set to the current timestamp.

## AUTO\_INCREMENT

Suppose:

```
StudentID INT AUTO_INCREMENT
```

Then:

```
INSERT INTO Students(Name)  
VALUES( 'Aadya' );
```

Result: **StudentID = 1**

Next:

```
INSERT INTO Students(Name)  
VALUES( 'Arpit' );
```

Result: **StudentID = 2**

## Explicit AUTO\_INCREMENT Value

You can still do:

```
INSERT INTO Students  
VALUES (100, 'Aadya');
```

The next generated value will continue after the highest existing value

## INSERT FROM SELECT

Copies rows from another table.

Suppose:

```
ArchiveStudents
```

```
INSERT INTO ArchiveStudents
SELECT *
FROM Students;
```

Every row is copied.

Copy only specific rows:

```
INSERT INTO ArchiveStudents
SELECT *
FROM Students
WHERE CGPA > 9;
```

Only high-CGPA students are archived.

Copy selected columns:

```
INSERT INTO StudentBackup(Name,Email)
SELECT Name,Email
FROM Students;
```

## INSERT IGNORE (MySQL)

Suppose:

```
Email UNIQUE
```

Existing: `aadya@example.com`

Now:

```
INSERT INTO Students
VALUES (...,'aadya@example.com');
```

Error: **Duplicate entry**

Instead:

```
INSERT IGNORE INTO Students
VALUES (...,'aadya@example.com');
```

Duplicate rows are skipped with a warning instead of aborting the statement.

## Common INSERT Errors

- Column Count Doesn't Match
- Wrong Datatype

```
INSERT INTO Students(Age)
VALUES( 'Twenty');
```

- Duplicate Primary Key
- NULL Violation

```
INSERT INTO Students(Age)
VALUES(20);
```

When: **Name NOT NULL**  
Error.